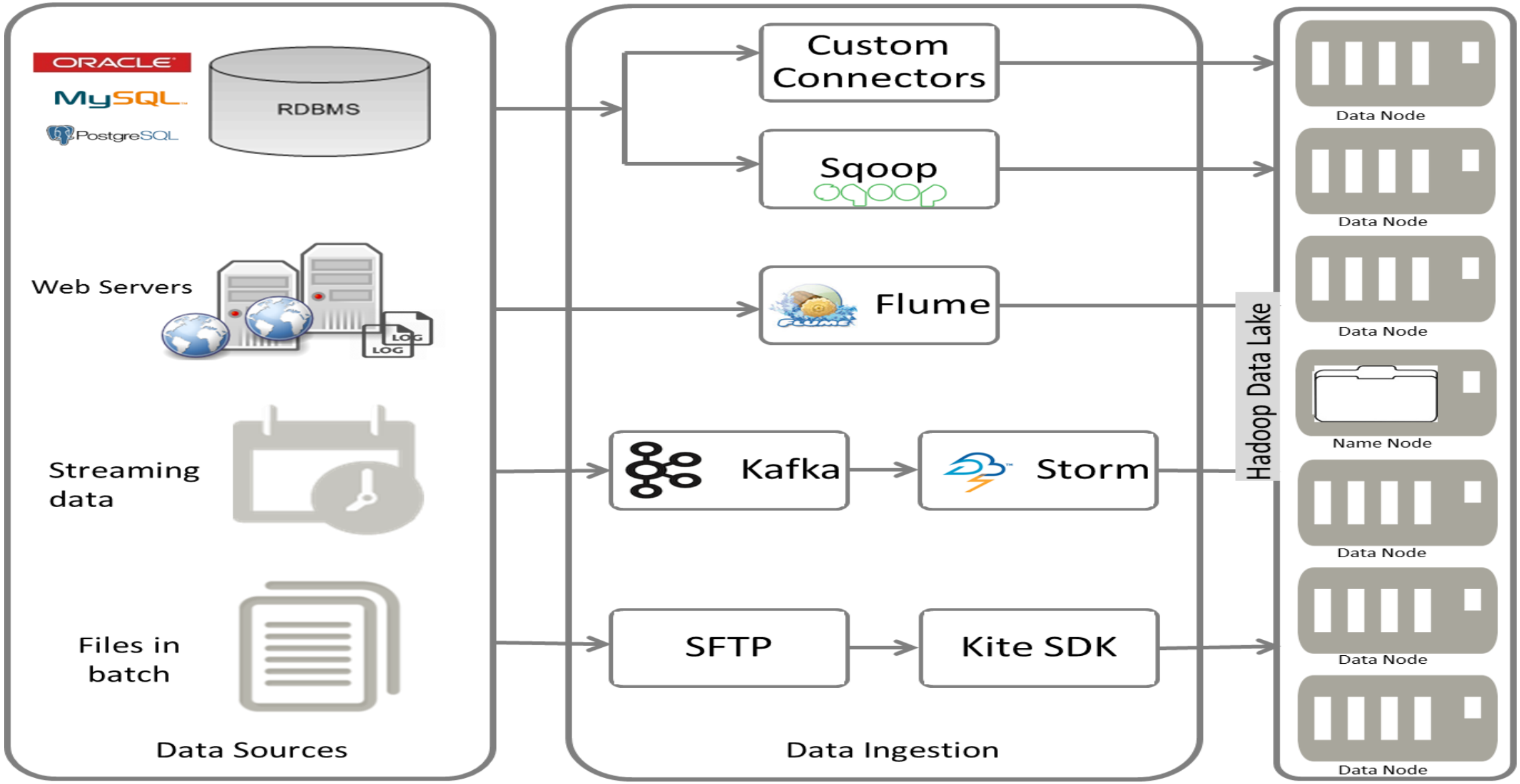
ТЕХНОЛОГИИ ХРАНЕНИЯ БОЛЬШИХ ДАННЫХ

Apache Spark

Загрузка данных в Hadoop



Apache Spark

- **Быстрая, выразительная кластерная вычислительная система, совместимая с Apache Hadoop**
- **Работает с любой системой хранения, поддерживаемой Hadoop (HDFS, S3, Avro,…)**
- **Повышает эффективность благодаря:**
  - **Примитивам вычислений в памяти**
  - **Графикам вычислений**
- **Улучшает удобство использования благодаря:**
  - **Богатым API в Java, Scala, Python**
  - **Интерактивной оболочке**



Ключевая идея Spark

- **Работайте с распределенными коллекциями, как с локальными**

- **Концепция: устойчивые распределенные наборы данных (RDD)**
- **Неизменные коллекции объектов, распределенных по кластеру**
- **Построены через параллельные преобразования (map, filter, …)**
- **Автоматически перестраивается при сбое**
- **Управляемое сохранение (например, кеширование в оперативной памяти)**

Действия в Spark

- **Transformations (e.g. map, filter, groupBy, join)**
  - **Операции, которые строят RDD из других RDD**
- **Actions (e.g. count, collect, save)**
  - **Возвращают результат или записывают его в хранилище**

Действия в Spark

- **Автономные программы могут быть написаны на любом из трех языков (Java, Scala, Python), но консоль - это только Python или Scala**
- **Python подходит для обоих вариантов**
- **Производительность: Java / Scala будет быстрее (статически типизированной), но Python лучше для вычислений с NumPy**

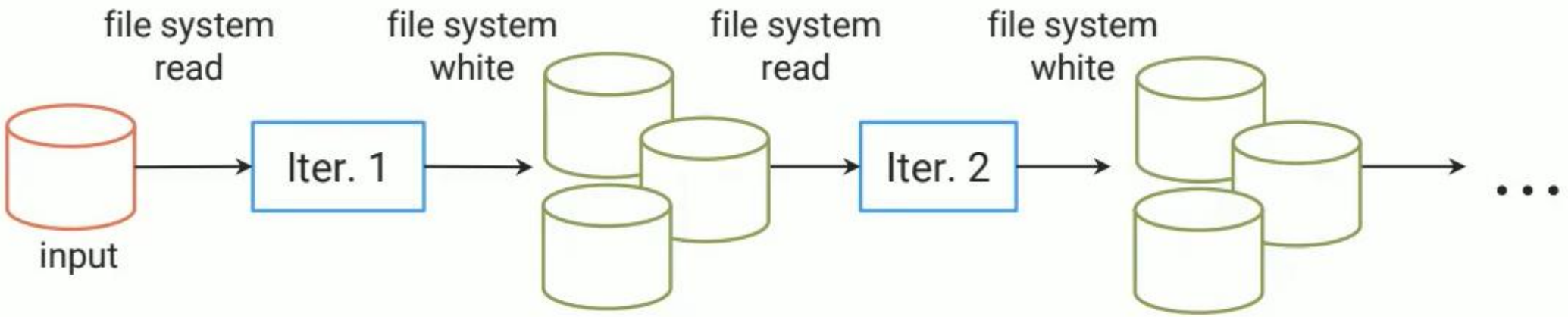
Spark Context

- Главная точка входа в функциональность Spark
- Создан в Spark Shell как переменная sc
- В автономных программах вы создаете свои собственные контексты

Поддержка языков в Spark

- **API в Java, Scala и Python**
- **Интерактивные оболочки в Scala и Python**

Ограничения Hadoop MapReduce



- Большие задержки
- Проблема с многопроходными алгоритмами
- Требуются внешние зависимости
- Отсутствие эффективных примитивов для разделения данных между процессами

Возможности и применение Apache Spark

Движок исполнения DAG (directed acyclic graph)

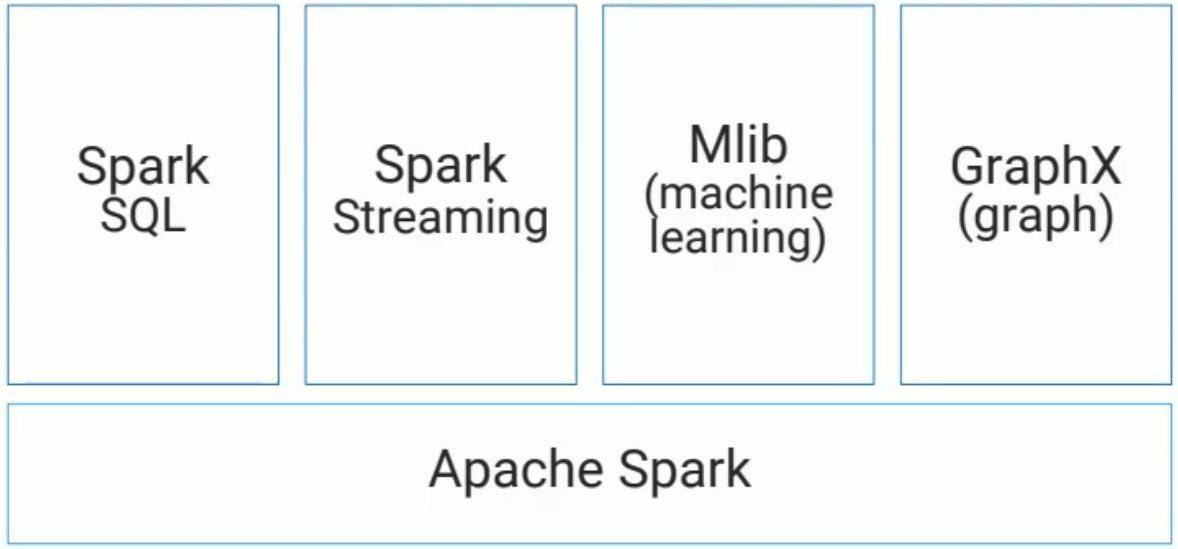


Запускается на множестве платформ

Встроенные средства доступа к

Удобный API для

Framework Apache Spark

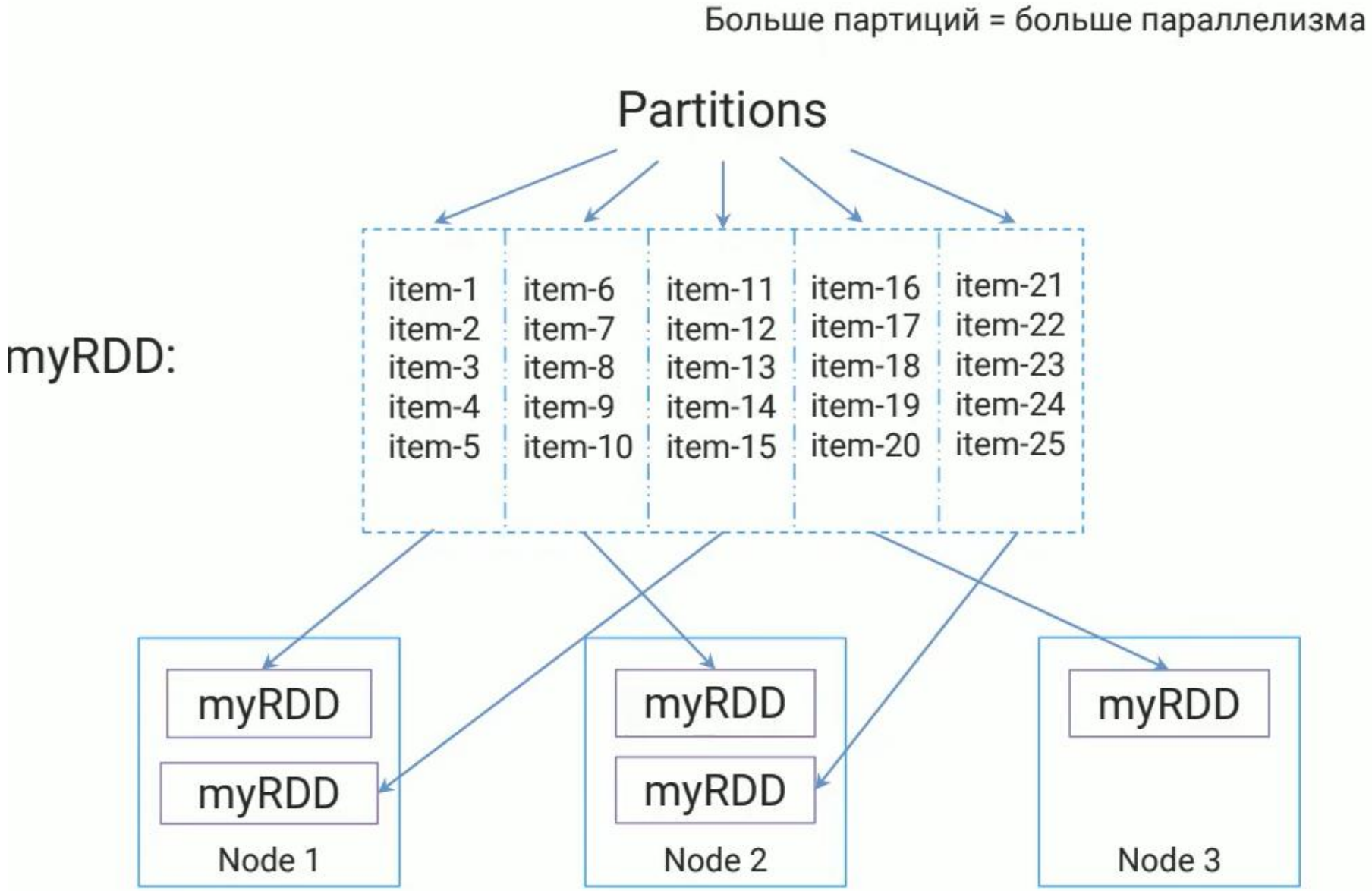


- RDD (Resilient Distributed Dataset)
- Spark SQL
- Spark Streaming
- Mllib
- GraphX

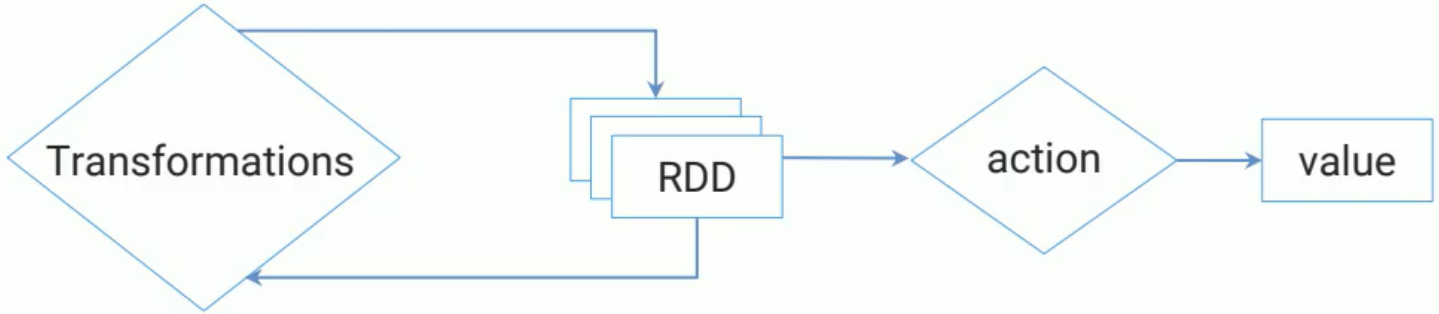
RDD (Resilient Distributed Dataset)

- Resilient Distributed Dataset - “устойчивый распределенный набор данных”
- Распределенная коллекция объектов на диске, в памяти, в HBase/Cassandra
- Неизменяемая коллекция объектов, распределенных по кластеру
- Управляемое пользователем распределение и хранение (память, диск)
- Статически типизированные (RDD[T])
- Построен с помощью параллельных преобразований
- Автоматически перестраивается в случае сбоя

Партиционирование RDD



Типы RDD операций



Трансформации :

- «ленивые» операции над RDD, которые создают один или несколько RDD
- не изменяют входной RDD
- пример: map, filter, reduceByKey, join

Действия :

- Запускает выполнение RDD-операций для получения значений
- пример : collect, count, take

Компоненты Spark

Driver

- Создает SparkContext и запускает основную функцию пользователя
- Выполняет различные параллельные операции на кластере

Executor

- Запускается на Worker node
- Запускает задачи и хранит данные в памяти или на диске

RDD

- Может быть создан 2 способами
  - Из коллекции (parallelize)
  - Считав данные из внешних источников «Ленивое» выполнение операций
- Может быть сохранен в памяти для переиспользования

Способы запуска Spark программ

Local mode



Standalone

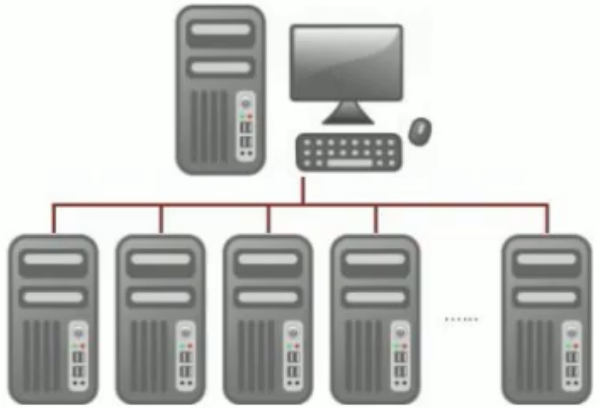
- Datastax

Spark On YARN

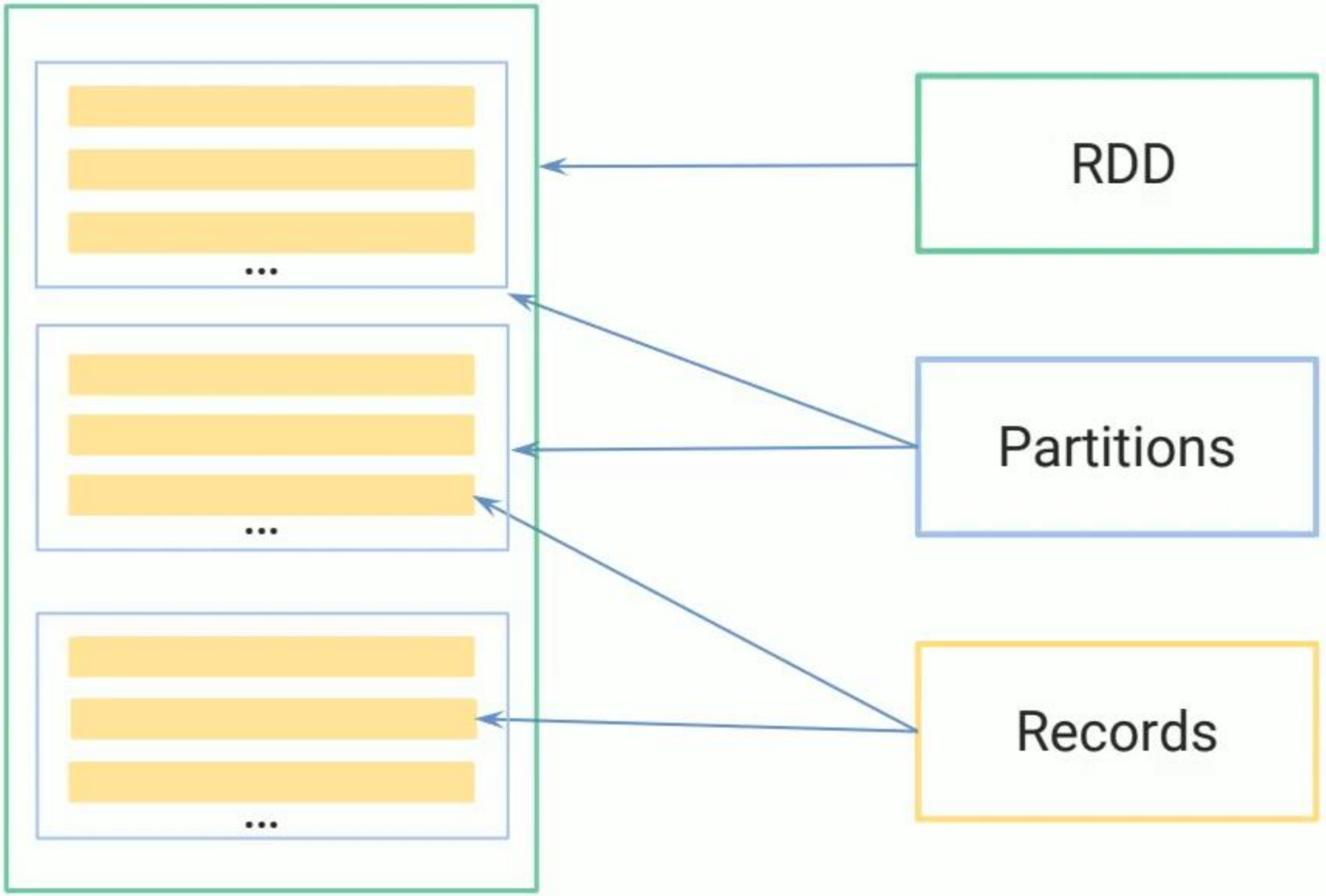
- Cloudera, Hortonworks, etc

Spark On Mesos

Spark on Kubernetes

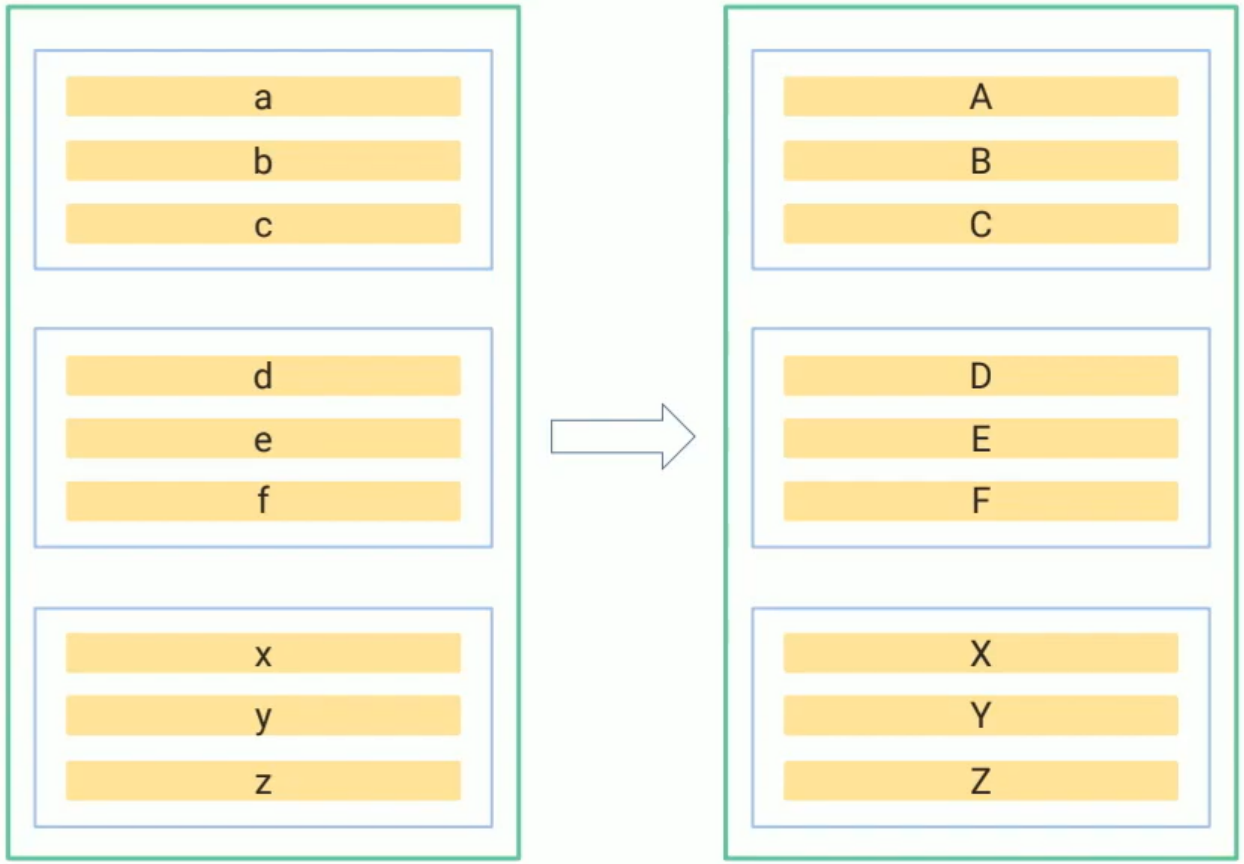


Операции над RDD



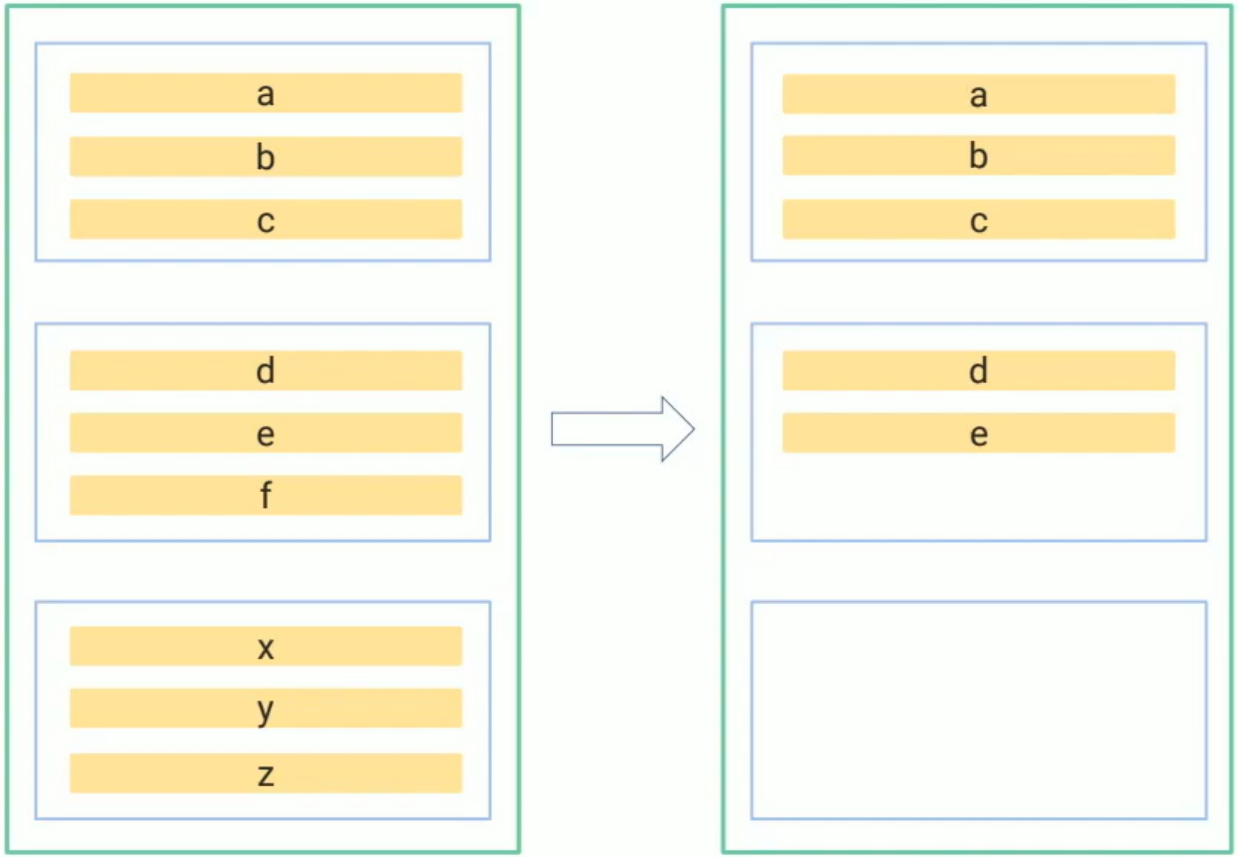
Трансформация Map с функцией UCASE

Возвращает новый распределенный набор данных, сформированный в результате применения функции трансформации к каждому элементу входных данных.



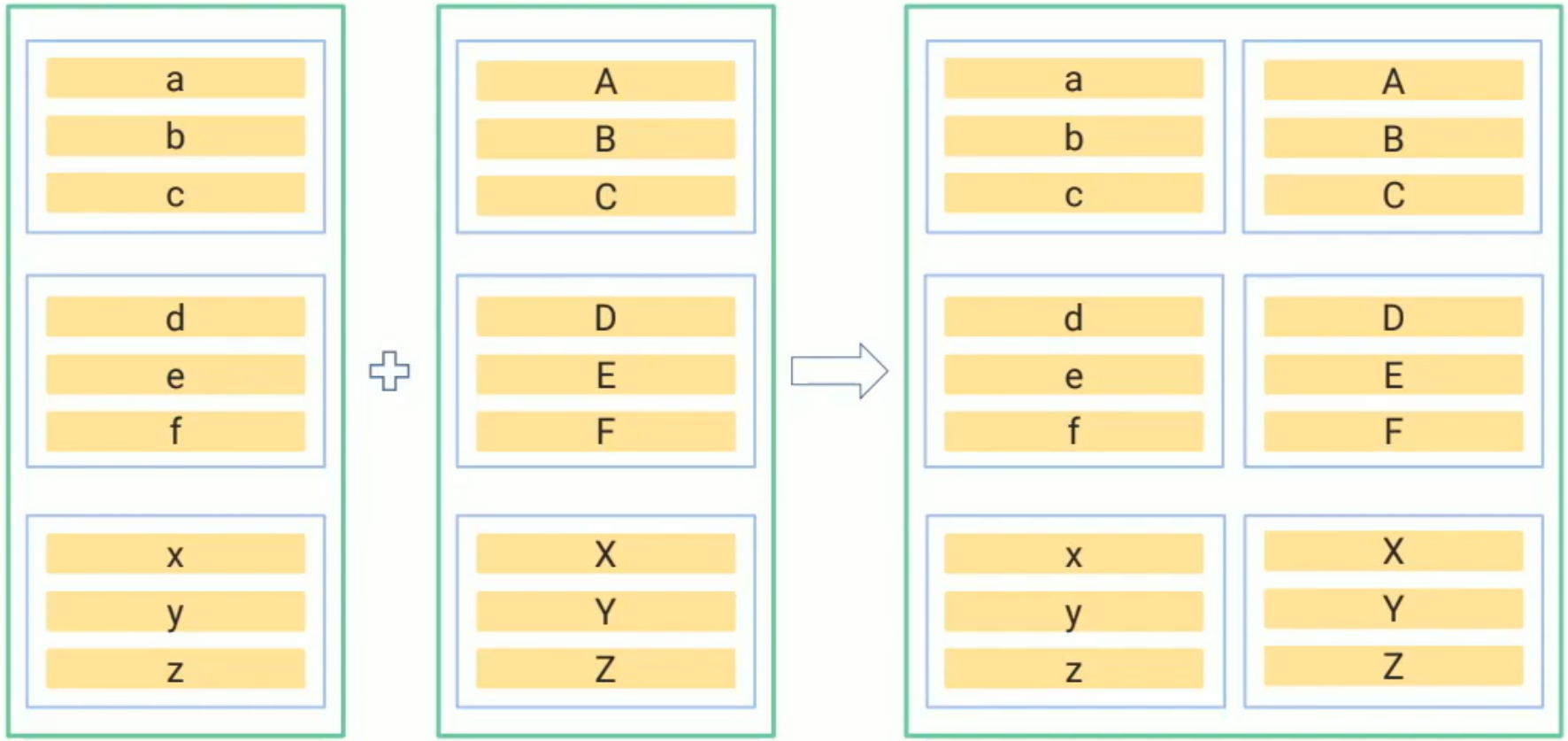
Трансформация Filter

Возвращает новый набор данных, сформированный из тех элементов источника, для которых критерий отбора возвращает значение true.



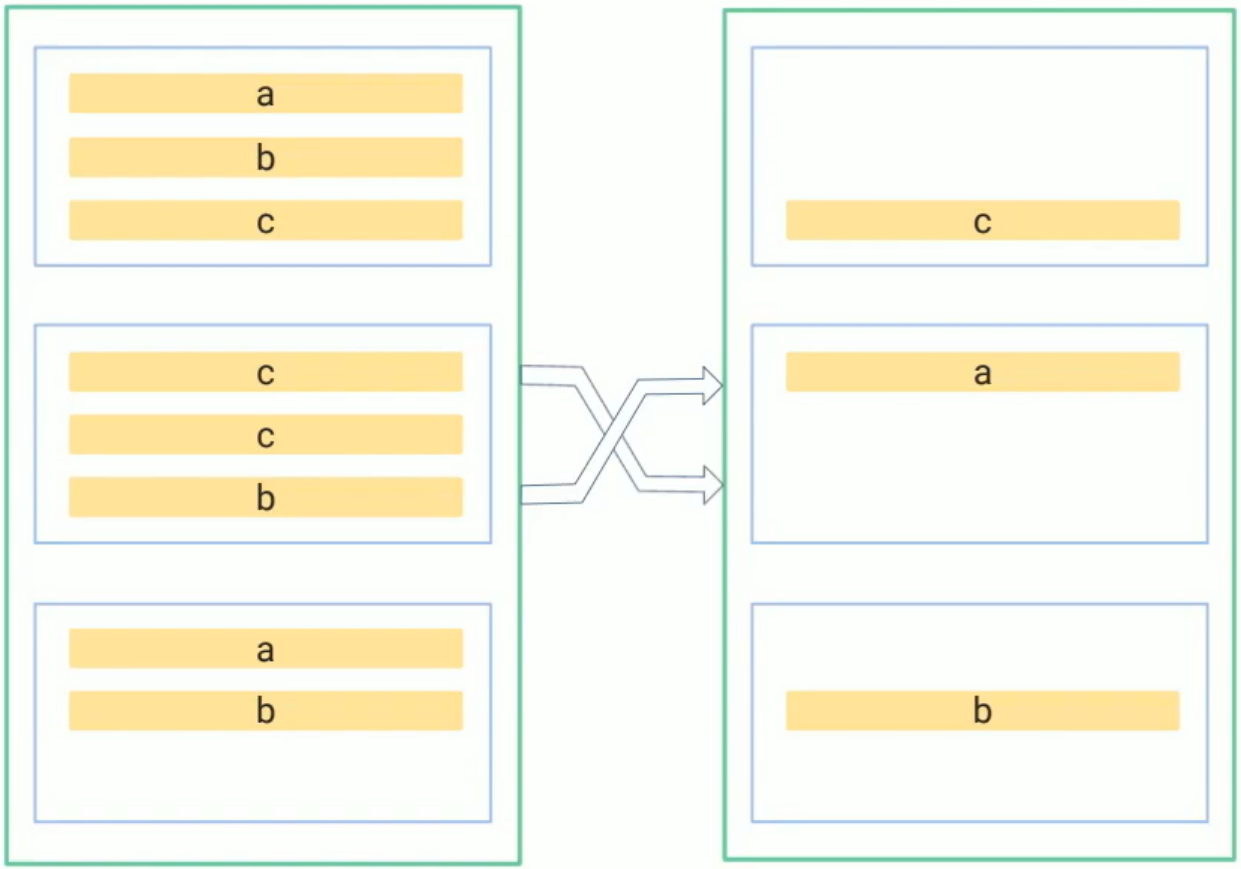
Трансформация Union

Возвращает новый набор данных, содержащий объединение элементов в исходном наборе данных и аргументе.



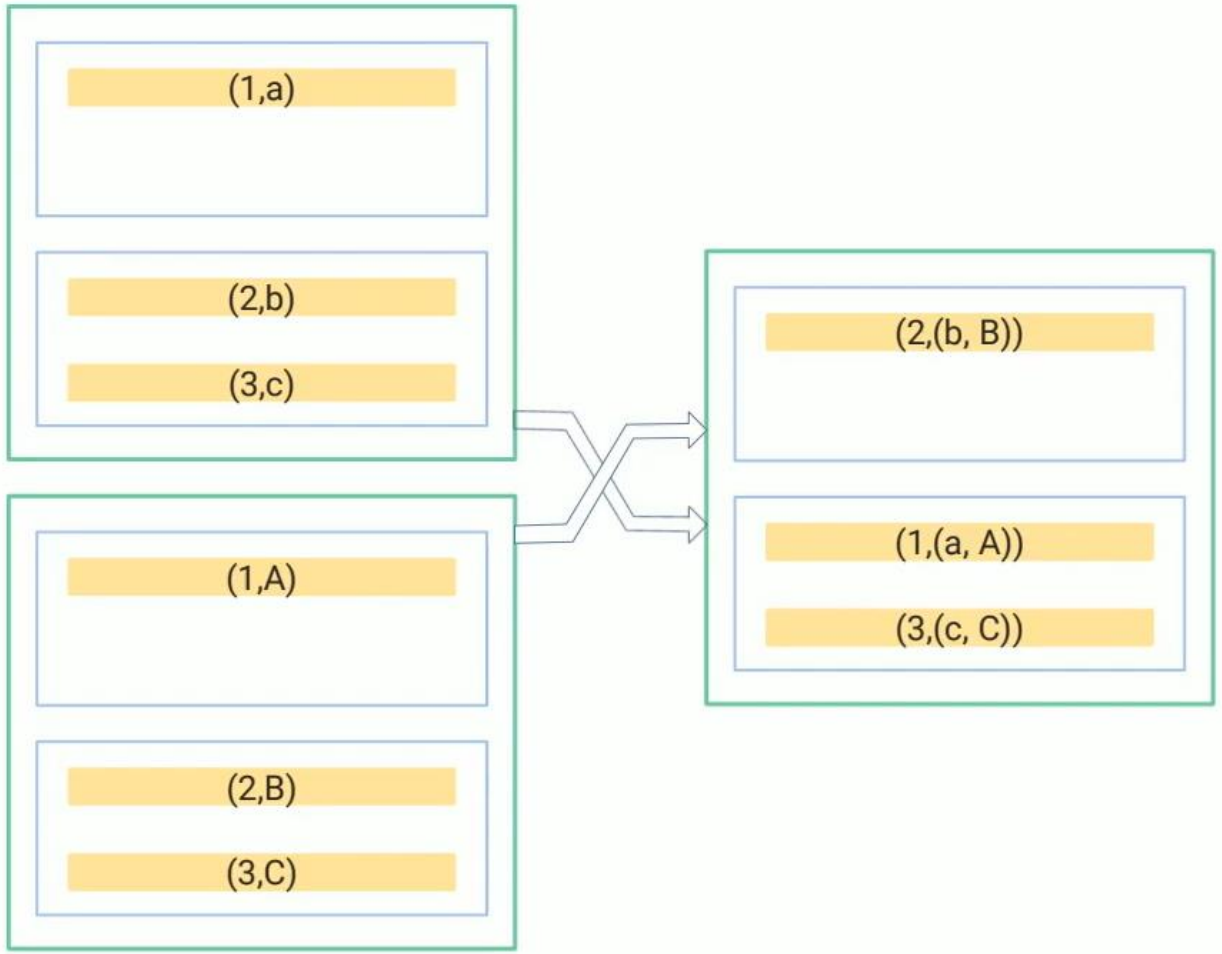
Трансформация Distinct

Возвращает новый набор данных, который содержит только уникальные элементы.



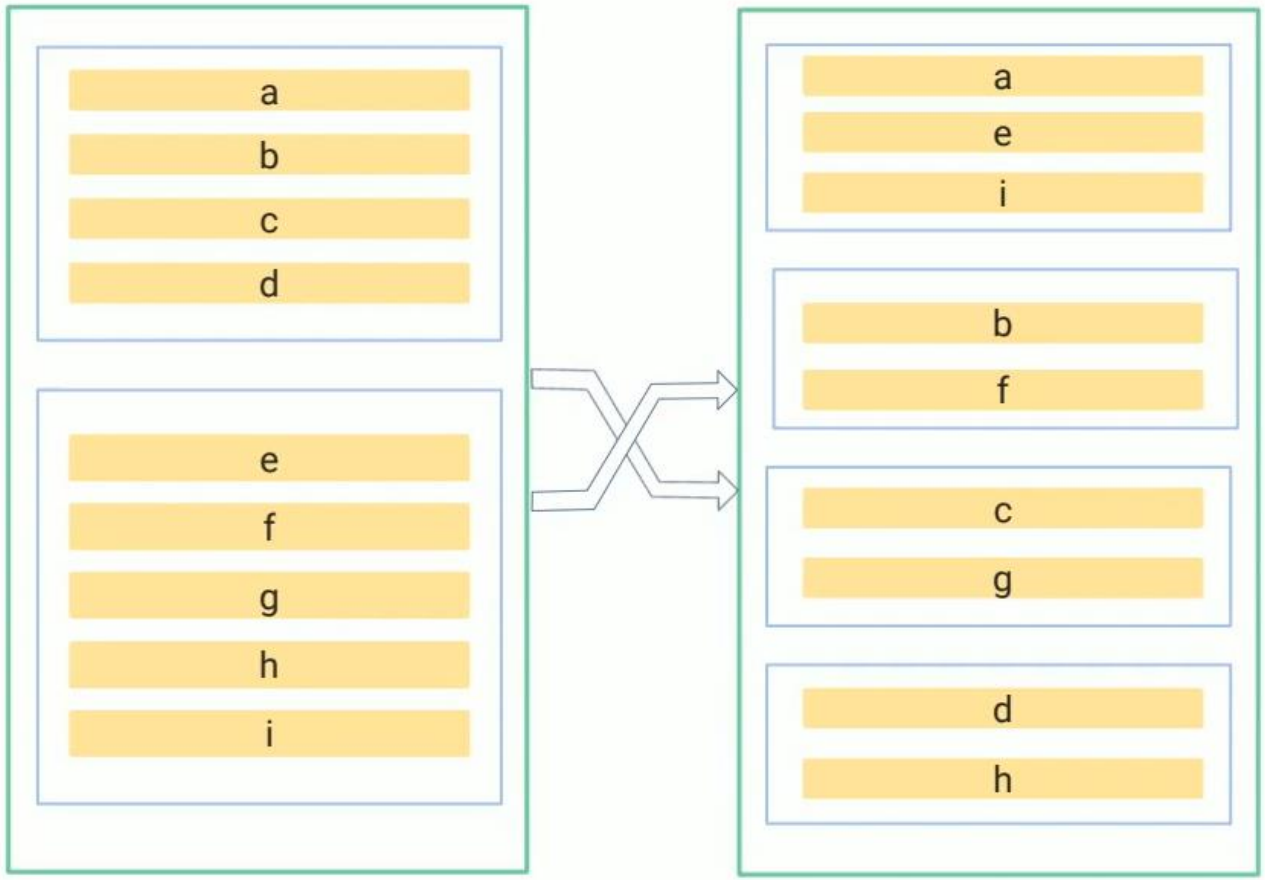
Трансформация Join

Операция объединения совмещает элементы на основе ключа, значение - кортеж с двумя полями исходного значения.

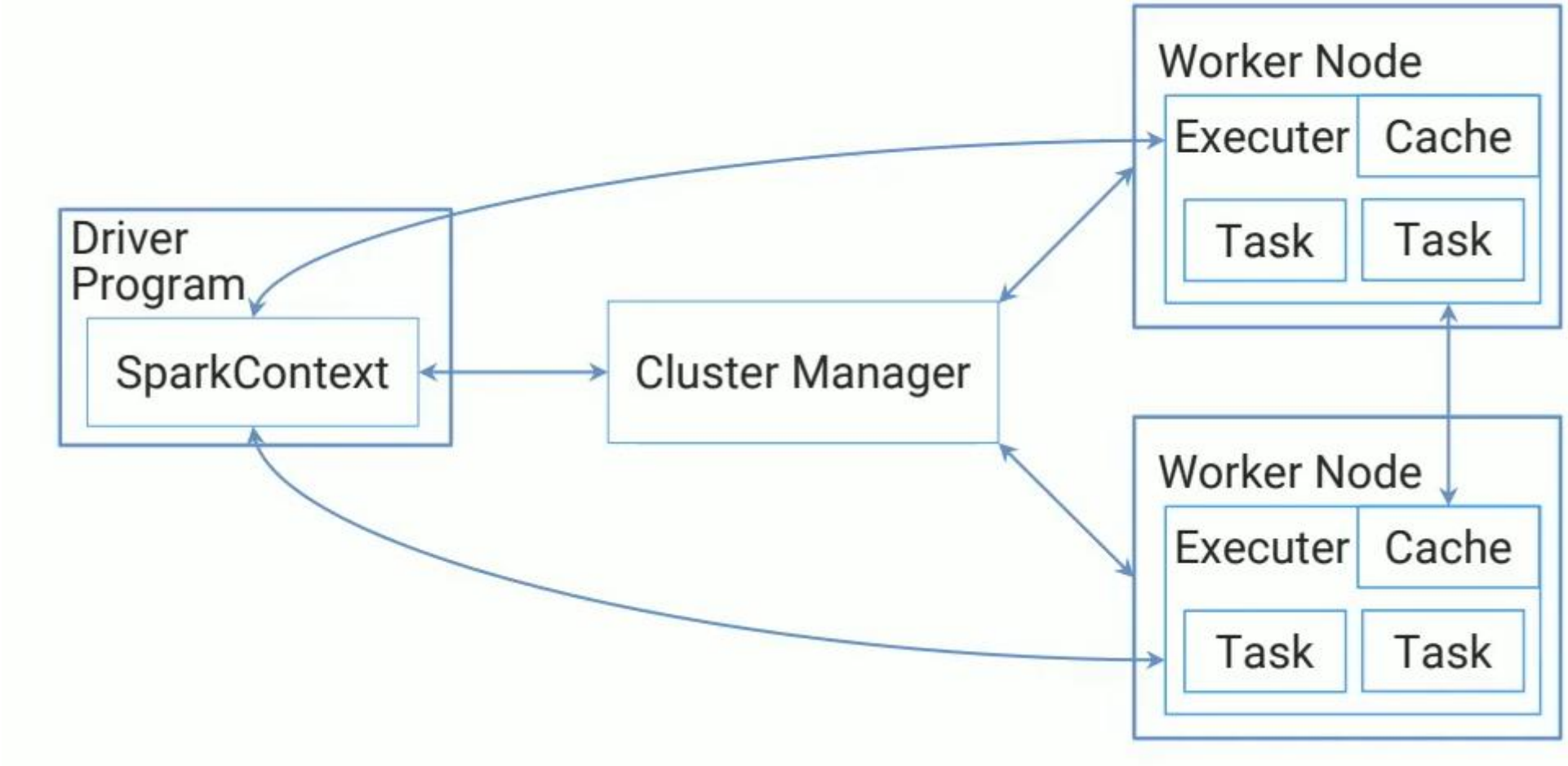


Трансформация Repartition

Произвольно переставляет данные в RDD, чтобы изменить количество разделов в большую или меньшую сторону.

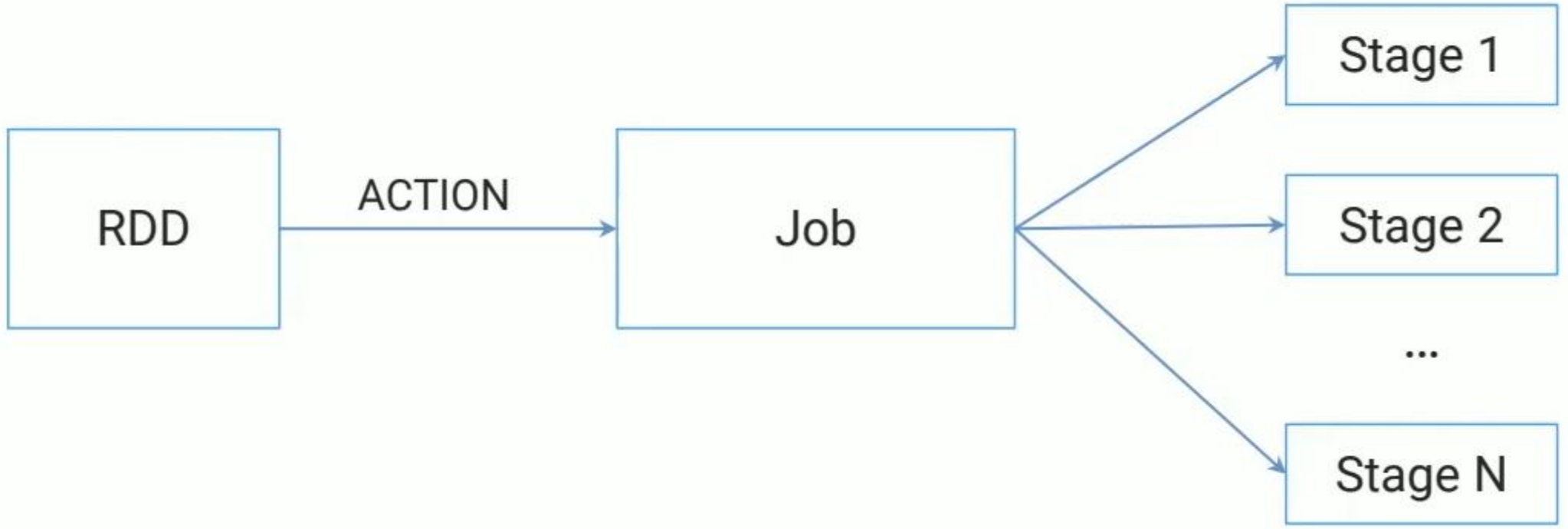


Компоненты кластера Spark



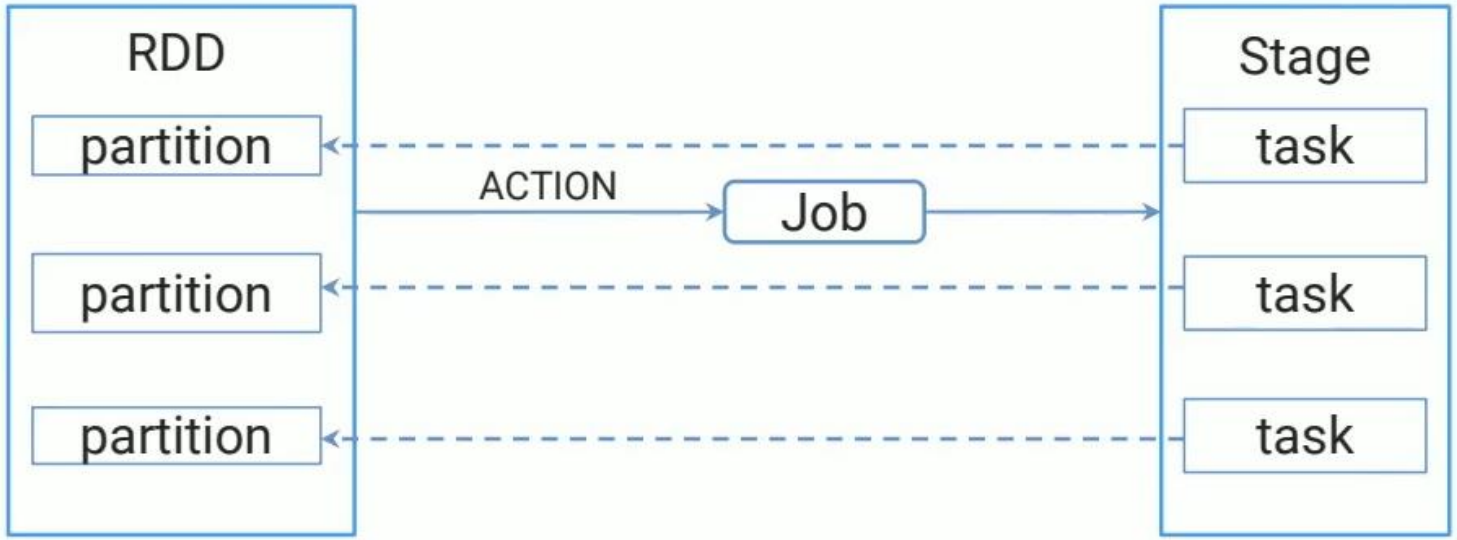
Job

Job – создается из RDD в результате наступления Action. Это рабочий элемент верхнего уровня, разбитый на стадии и переданный в DAGSheduler



Стадия (Stage)

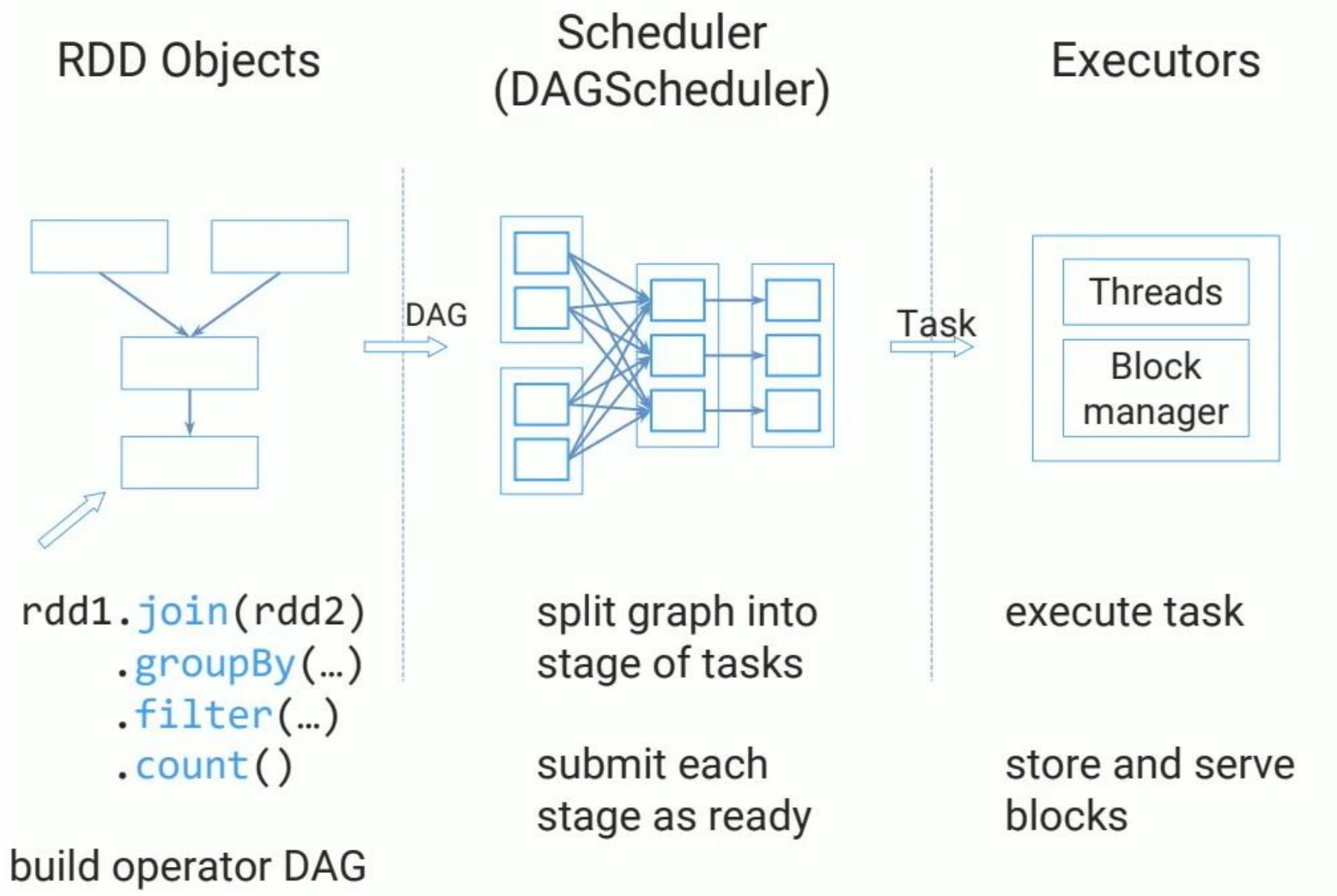
- Набор параллельных задач, которые вычисляют частичные результаты функции
- По одной стадии на партицию RDD
- Конвейеризируется и запускается одним независимым исполнителем



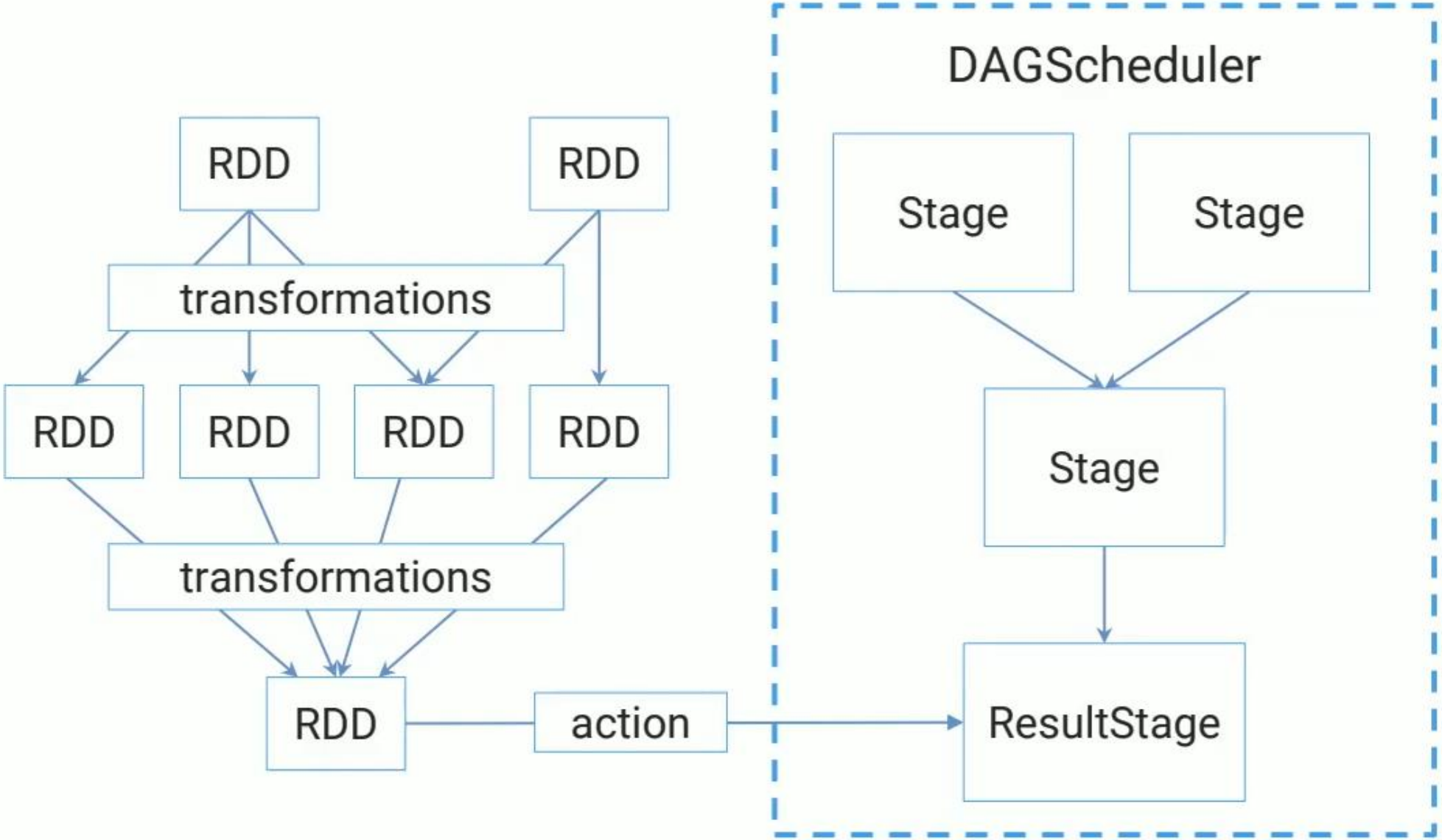
Задача (Task)

- Вычисление над партицией данных для job stage
- Состояния задачи:
  - LAUNCHING
  - RUNNING
  - FINISHED
  - FAILED
  - KILLED
  - LOST

Планирование Job



Dag Scheduler



Spark Streaming

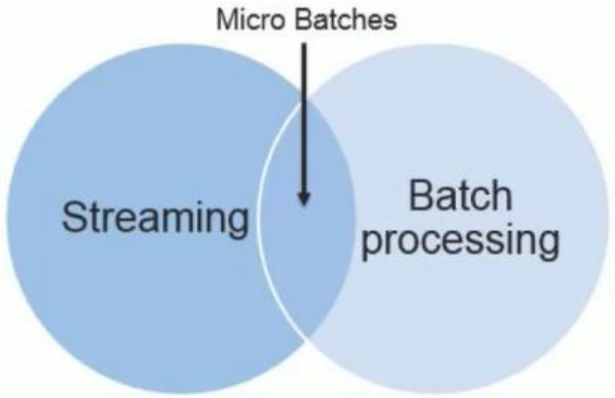
- Предоставляет способ использовать непрерывные потоки данных
- Масштабируемый, высокопроизводительный, отказоустойчивый
- Построен на основе Spark RDD
- API похож на Core API
- Поддерживает много входов
- Интегрируется с другими модулями Spark
- Микропакетная архитектура



Микропакетная архитектура Spark Streaming

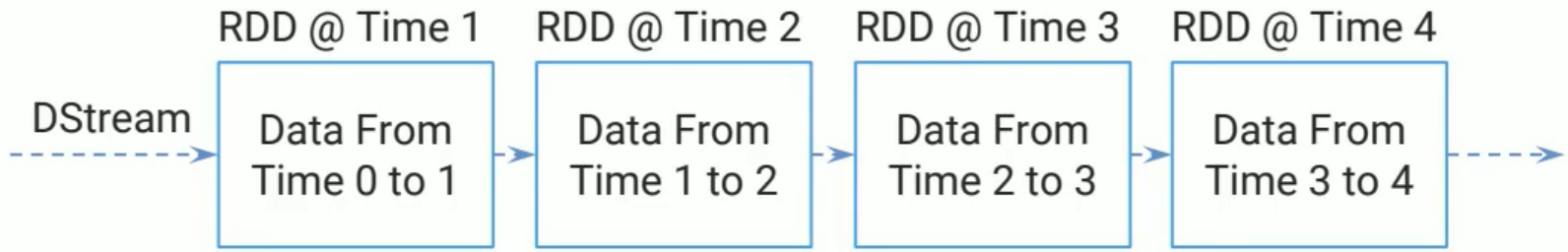
- Постоянно получает входные потоки данных
- Делит данные на небольшие, детерминированные пакетные задания
- Пакеты обрабатываются Spark Engine



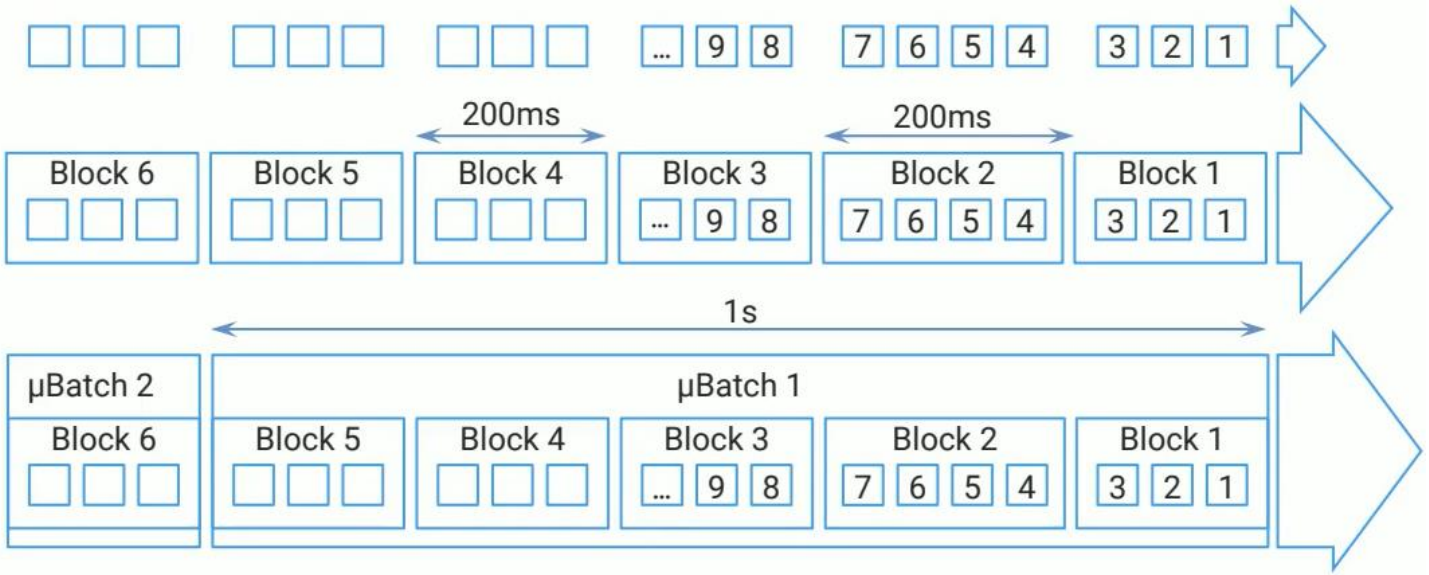


Модель программирования Spark Streaming

- DStream = Discretized Stream
- DStream - это серия пакетов
- Каждый пакет представлен как RDD
- Каждый RDD содержит данные за определенный интервал
- Операции DStream приводят к преобразованию RDD
- Возможность прямого доступа к API RDD
  - Объединиение потоковой и пакетной обработки
- Основные метрики
  - Интервал между пакетами
  - Интервал между блоками



Spark Streaming



- Интервал между блоками
  - Входные данные разбиваются на блоки
  - Один блок создается за интервал между блоками
  - По умолчанию 200 мс, конфигурируемый
- Интервал между пакетами
  - Новые пакеты создаются через равные промежутки времени
  - В конце каждого интервала создается новый RDD
  - Обычно от 500 мс до нескольких секунд

Spark Streaming приложение

- StreamingContext является точкой входа для всех функций потоковой передачи
- Переносит / расширяет существующий контекст Spark

```
val ssc = new StreamingContext(sc, Seconds (1))
```

- Программирование
  - Определяем входные источники

```
val lines = ssc.socketTextStream(...)
```

  - Определяем потоковые вычисления

```
lines.map(_.toLowerCase).print
```

  - Начинаем получение и обработку данных

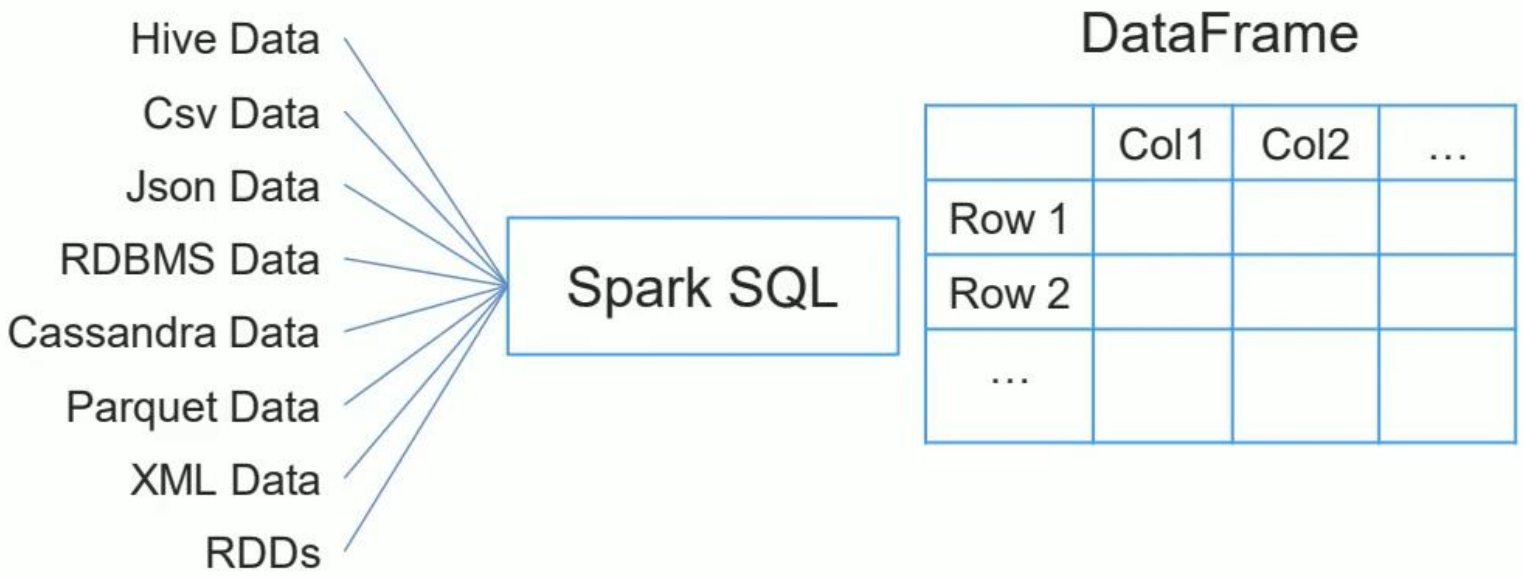
```
ssc.start()
```

  - Ожидаем окончание процесса

```
ssc.awaitTermination()
```

Spark SQL: Data Frame

- Распределенный сбор данных, организованный в именованные столбцы
- Концептуально эквивалентно таблице RDMBS
- API в декларативном стиле или SQL
- Автоматическая оптимизация
- Доступно в Python, Scala, Java, R



Spark SQL: код

RDD

```
rdd.map(x => (x.dept, (x.age, 1)))
   .reduceByKey {case (x,y) => (x._1+y._1, x._2+y._2)}
   .mapValues { case (x,y) => x / y }
```

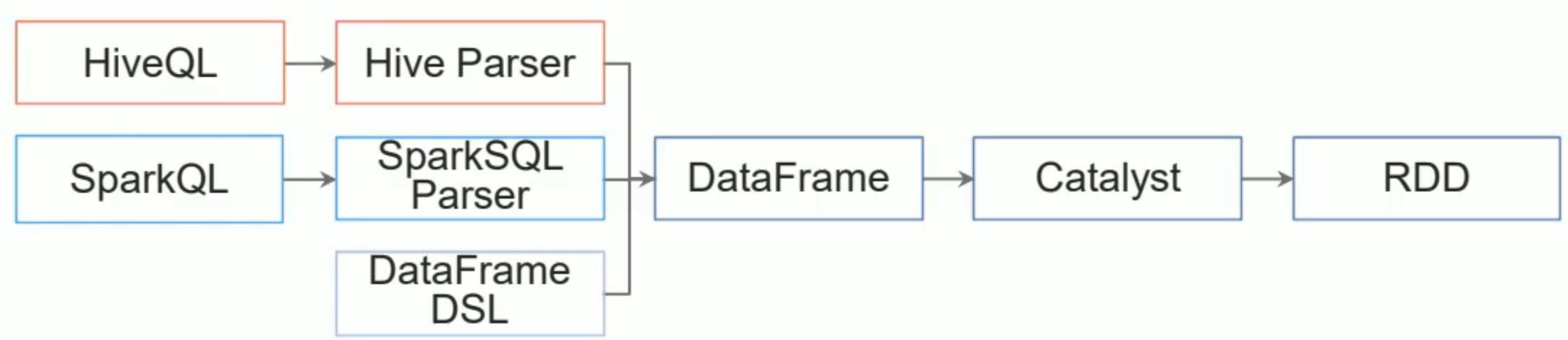
DataFrame

```
df.groupBy("dept"))
   .avg("age")
```

SQL

```
sqlContext.sql("SELECT avg(age)" +
   "FROM people GROUP BY dept")
```

Spark SQL pipeline



DataSource API

- Универсальный API для загрузки и сохранения структурированных данных
- Встроенная поддержка
  - Parquet, JSON, Hive, JDBC
- Сторонние пакеты
  - https://github.com/databricks
  - AVRO, CSV, XML...
- Конфигурируемый

Создание Data Frame

- Структура данных DataFrame описывается схемой
- Схема должна быть выведена
  - Через API источника данных
    - Автоматический вывод схемы
  - Из RDD
    - При помощи рефлексии
    - Программно
- Совместимость с RDD